SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:100**

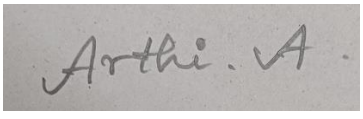
Annexure A

Scenario based

Code of Conduct:

*I **ARTHI A – 192525152** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.

4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.

2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:
• P1 – Emergency patient registration (Highest Priority)
• P2 – Laboratory report generation (Medium Priority)
• P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks
1. Identify the scheduling issue occurring in the system.
2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.
4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:
• Incorrect stock counts
• Duplicate order confirmations
• Frequent data inconsistencies

Tasks
1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:
• 60,000 users connect simultaneously.
• CPU utilisation reaches 95%.
• Memory consumption increases rapidly.
• Video buffering becomes frequent.

Tasks
1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock

A smart city traffic management system controls four intersections.

Resources:
• R1 – North Junction
• R2 – East Junction
• R3 – South Junction
• R4 – West Junction

Current situation:
• T1 holds R1 and requests R2.
• T2 holds R2 and requests R3.
• T3 holds R3 and requests R4.
• T4 holds R4 and requests R1.

Tasks
1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation

An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:
• Thousands of students log in simultaneously.
• New resource requests arrive continuously.
• The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks
1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

1. Most suitable scheduling algorithm

Pre-emptive Priority Scheduling with aging is the best fit. The three categories have clearly different importance (money first, availability second, printing last), which priority scheduling

expresses directly; aging is added so the low-priority confirmation jobs are not postponed indefinitely.

Algorithm	**Assessment for this system**
FCFS	Convoy effect: a 20 ms job blocks 2 ms payments. Ignores importance.
SJF	Good average wait, but burst times must be predicted and long jobs (printing) starve. Ignores importance.
Round Robin	Fair, but treats payment and printing equally, so payments wait behind other work.
Priority (pre-emptive) + aging	Payments run immediately, seat checks next, confirmations are guaranteed progress by aging. **Selected.**

2. Why it improves CPU utilization and customer wait time

- **Short, critical work first:** payment (P1) pre-empts anything running, so the most time-sensitive step never queues behind printing, and the payment gateway session is released quickly.
- **Better utilization:** because P1/P2 jobs frequently block on network/DB I/O, the CPU is immediately handed to the next-highest ready process instead of idling; the CPU stays busy while I/O completes.
- **Confirmation delay fixed by aging:** every 10 ms a process waits, its priority is raised one level. A waiting P3 job eventually reaches P2/P1 level and runs, so the complaint (slow confirmations) disappears without hurting payments.
- **Lower average waiting time** than FCFS in the mixed workload (see the worked example below).

3. Sample execution order

Assumed jobs: P1a (arrives 0, burst 4 ms), P2a (0, 8 ms), P3a (0, 12 ms) and P1b (arrives at 6 ms, burst 2 ms). Smaller number = higher priority: P1 = 1, P2 = 2, P3 = 3.

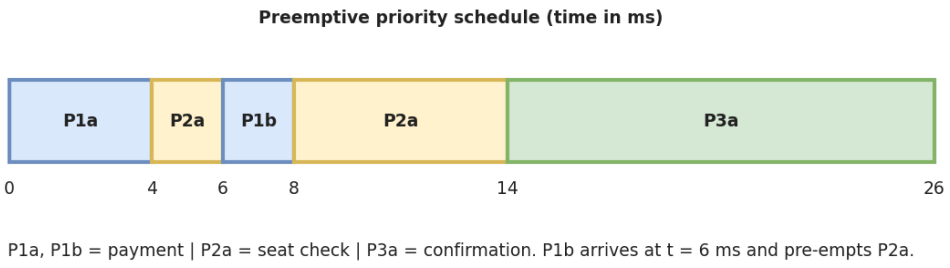


Figure 1 – Gantt chart (draw.io: q1_gantt_priority.drawio)

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1a (payment)	0	4	4	4	0
P1b (payment)	6	2	8	2	0
P2a (seat check)	0	8	14	14	6
P3a (confirmation)	0	12	26	26	14

Average waiting time = (0 + 0 + 6 + 14) / 4 = **5 ms**. Execution order: P1a → P2a (2 ms) → P1b (pre-empts) → P2a (rest) → P3a. With aging, if P3a had waited past the aging threshold it would be boosted and run earlier.

4. Drawback in real-time systems

Starvation / unbounded delay and priority inversion. Without aging, continuous arrival of payment requests during festival peaks can starve confirmations; even with aging the worst-case wait of a low-priority job is hard to bound, which is a problem when hard deadlines must be guaranteed. If a low-priority job holds a lock needed by a high-priority one, priority inversion occurs (mitigated by priority inheritance).

Question 2 — Deadlock in a Banking Transaction Server

1. Text-based Resource Allocation Graph

Notation: T --> R : request edge R --> T : assignment (R held by T)

A --> T1 --> B --> T2 --> C --> T3 --> A

i.e. A held by T1, T1 requests B,
B held by T2, T2 requests C,
C held by T3, T3 requests A (cycle closes at A)

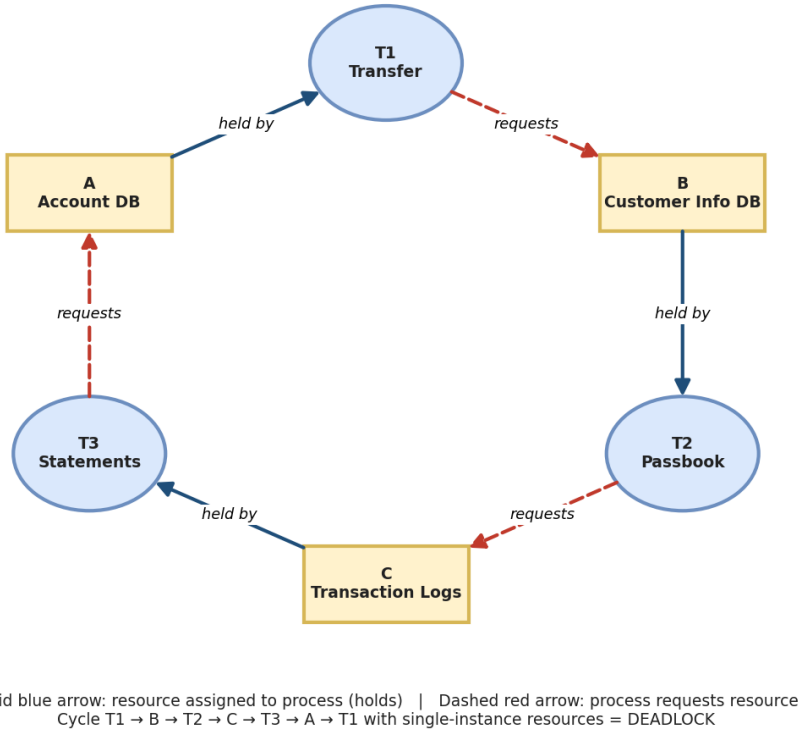


Figure 2 – Resource Allocation Graph (draw.io: q2_rag_banking.drawio)

2. Has a deadlock occurred?

Yes. The RAG contains the cycle T1 → B → T2 → C → T3 → A → T1. Each resource (A, B, C) has a single instance, so a cycle is both necessary **and sufficient** for deadlock. All four Coffman

conditions hold: mutual exclusion (database locks are exclusive), hold-and-wait (each transaction holds one lock and requests another), no pre-emption (locks are not forcibly taken) and circular wait (the cycle above). No transaction can proceed, so none will ever release its lock.

3. Recommended technique

Deadlock prevention by breaking circular wait (resource ordering). Every lock is given a global rank A < B < C and transactions may request locks only in increasing rank. A request that violates the order is not allowed; the transaction must release and re-acquire in order.

How it fixes the issue: T3 currently holds C and asks for A (rank 1 after rank 3) – this would be rejected, so T3 would have acquired A first and then C. A cycle needs at least one 'backward' edge from a higher-ranked to a lower-ranked resource; ordering makes that impossible, so circular wait can never form.

4. Redesign of the locking order

- **Global order: A (Account DB) → B (Customer Info DB) → C (Transaction Logs).**
- T1 (transfer): lock A, then B. T2 (passbook): lock B, then C. T3 (statements): lock A, then C – A must be taken first even if it is needed later.
- T3 is read-only: run it on a **consistent snapshot** (MVCC / read-committed replica) so it holds no write locks on A, B or C at all.
- Use short critical sections, `tryLock` with a timeout and retry-with-back-off as a safety net, and log lock-wait times to detect regressions.

Question 3 — Mobile OS App Switching Lag

1. Why the CPU is idle despite many active processes

- **Most 'active' apps are not runnable.** Background apps are mostly in the waiting state (network, touch, timers, disk). Only a few are in the ready queue, so there is little CPU work at any instant.
- **I/O and memory stalls.** Launching WhatsApp triggers storage reads and page faults; the CPU idles while flash I/O and page-ins complete. With 20 apps resident, memory pressure/low-memory-killer activity increases these stalls (similar to thrashing).
- **Scheduling latency, not shortage of CPU.** Round Robin gives every ready app a full turn in order, so the audio thread waits behind other apps even though the CPU overall is lightly loaded; idle time and lag therefore appear together.
- **Convoy effect:** an app that uses its full quantum delays the I/O-bound apps that would otherwise issue their next I/O quickly and keep the devices busy.

2. Is the 20 ms quantum helping or hurting?

It **hurts** latency-sensitive work. In Round Robin the worst-case wait for a ready process is about $(n - 1) \times q$. If all 20 apps are ready: $(20 - 1) \times 20$ ms = **380 ms**, far above the 10–20 ms an audio buffer or a 60 fps UI frame (16.7 ms) can tolerate, which causes music stutter and janky switching. The context-switch overhead of a 20 ms quantum is small (about 0.1 ms / 20 ms = 0.5 % assuming 0.1 ms per switch), so the large quantum buys almost nothing in efficiency while costing responsiveness.

3. Recommended scheduling method / quantum

Replace flat Round Robin with a **Multilevel Feedback Queue (MFQ)** that classifies tasks by behaviour and importance:

Queue	Tasks	Policy	Quantum
Q0 – real-time / audio	Music player audio thread, call audio, UI compositor	Fixed priority, pre-emptive	Short, deadline-driven
Q1 – foreground / interactive	App in front (WhatsApp, social apps)	Round Robin	**8–10 ms**
Q2 – background	Sync, downloads, idle apps	RR / batch, lower priority; frozen or throttled	20–40 ms

Tasks that use their whole quantum are demoted; tasks that block early (interactive, I/O-bound) stay high. Aging promotes starved background work. Background apps that are idle are frozen and removed from the run queue.

4. How this reduces lag and improves throughput

- **Latency:** with only ~4 runnable interactive apps and q = 8 ms, worst-case wait ≈ (4 − 1) × 8 = **24 ms**, versus 380 ms. Audio runs in Q0 and always pre-empts, so opening WhatsApp no longer delays the player.
- **Throughput:** I/O-bound apps issue their next I/O sooner and overlap with CPU-bound work, which raises device utilization; freezing idle apps removes useless context switches and cache pollution.
- **Overhead stays low:** with a 0.1 ms switch time, q = 8 ms costs about 1.2 %, an acceptable price for a large responsiveness gain.

Question 4 — Deadlock Risk in an Operating System Update

1. Can this sequence lead to a deadlock?

Yes. Each resource is single-instance, mutually exclusive and non-pre-emptible; every one of M1–M3 holds one resource while waiting for another, and the waits close a loop. This is an actual deadlock that has already formed, not merely a risk. M4 is not part of the cycle; it is blocked waiting for the others to finish, so it is a victim of the deadlock.

2. The exact circular wait

```
M1 --holds--> R1 --wanted by--> M3

M1 --waits for--> R2 --held by--> M2

M2 --waits for--> R3 --held by--> M3

M3 --waits for--> R1 --held by--> M1

 Wait-for cycle:  M1 -> M2 -> M3 -> M1
```

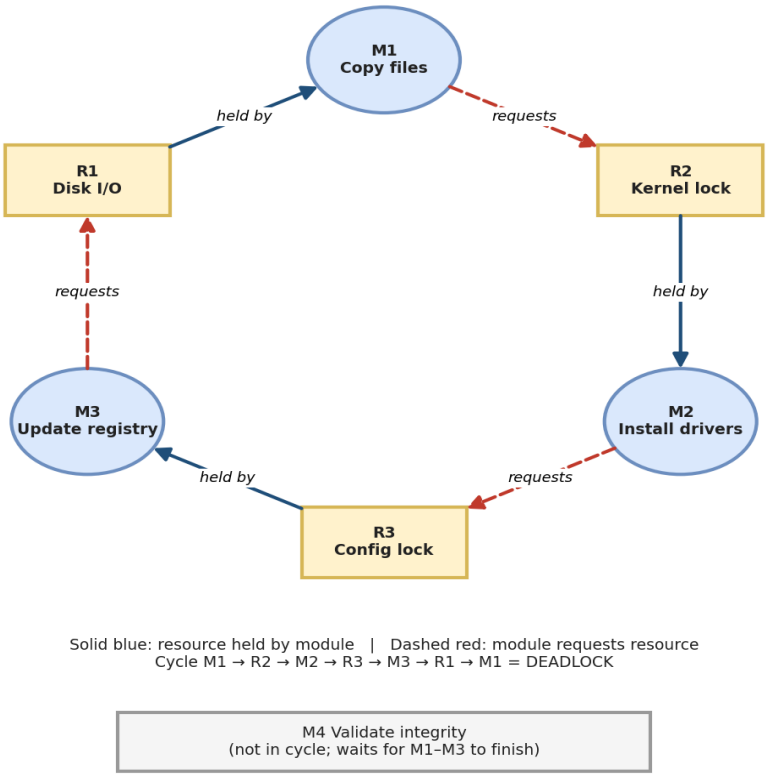


Figure 3 – Resource Allocation Graph (draw.io: q4_rag_os_update.drawio)

3. Banker's-algorithm-style allocation to avoid the risk

Each module declares its maximum need in advance (Max). The OS grants a request only if the resulting state is **safe**. Resources: R1 = R2 = R3 = 1 instance. Max claims: M1 = (R1, R2), M2 = (R2, R3), M3 = (R3, R1), M4 = (R1).

State after M1 is granted R1 and M2 is granted R2 (still safe):

Module	**Max (R1 R2 R3)**	**Allocation**	**Need**	**Available**
M1	1 1 0	1 0 0	0 1 0	0 0 1
M2	0 1 1	0 1 0	0 0 1	
M3	1 0 1	0 0 0	1 0 1	
M4	1 0 0	0 0 0	1 0 0	

Safe sequence check: Available (0 0 1) satisfies M2's need → M2 finishes, Available = (0 1 1) → M1 finishes, Available = (1 1 1) → M3, M4 finish. Safe: ⟨M2, M1, M3, M4⟩.

Now M3 requests R3. Pretend to grant it: Available becomes (0 0 0) and the needs are M1 (0 1 0), M2 (0 0 1), M3 (1 0 0), M4 (1 0 0). No need can be met, so the state is **unsafe** and the request is **denied** – M3 waits, so the circular wait never forms.

4. Prioritizing modules to improve CPU utilization

- **Give higher priority to modules holding critical locks for short periods** (M2 kernel lock, M3 registry/config lock) so they finish and release R2/R3 quickly; fewer waiters means fewer idle threads.
- **M1 (copy files) is I/O-bound:** give it a high CPU priority and a small quantum so it issues disk requests early; while the disk works, the CPU runs M2/M3, overlapping I/O and computation.
- **M4 (validation) runs last at lowest priority** because it depends on the others, avoiding wasted cycles on polling/blocking.
- Enforce a **global lock order R1 < R2 < R3** for all modules as additional protection, and checkpoint progress so a failed step can be rolled back.

Question 5 — Cloud Server CPU Utilization Analysis

1. CPU utilization

CPU utilization = $1 - p^n$ (probability that at least one VM is ready to run)

$= 1 - 0.8^7 = 1 - 0.2097 = 0.7903 \rightarrow \approx$ **79 %**

2. Interpretation

The server is **moderately well utilized but not optimal**. About 21 % of CPU time is wasted because at some instants all 7 VMs are simultaneously waiting for I/O. Cloud providers usually target ≥ 90 %: from the table below that needs about 11 VMs (91.4 %). So there is spare capacity, provided memory and I/O bandwidth allow more VMs.

3. Effect of changing the number of VMs

VMs (n)	**1**	**2**	**4**	**7**	**10**
Utilization	20.0 %	36.0 %	59.0 %	79.0 %	89.3 %

VMs (n)	**11**	**12**	**14**	**16**	**20**
Utilization	91.4 %	93.1 %	95.6 %	97.2 %	98.8 %

- **Increase VMs:** utilization and throughput rise, but with diminishing returns (from 7 to 8 VMs +4.2 points; from 14 to 15 only +0.9 points). Beyond a point, memory pressure (ballooning/swapping), cache contention and context-switch overhead can reduce real throughput, so the formula is an upper bound that ignores these costs.
- **Decrease VMs:** utilization falls quickly (4 VMs → 59 %), wasting hardware and raising cost per VM.

4. Two OS-level strategies

- **Raise the degree of multiprogramming with admission control and load balancing:** place more VMs per host until CPU utilization is ≈ 90 % while monitoring free memory and I/O queue depth; use live migration to move VMs from overloaded hosts.
- **Reduce I/O wait (lower p):** disk/page caching and read-ahead, asynchronous/paravirtual I/O (virtio) and a good I/O scheduler (deadline / multi-queue) with SSD/NVMe storage, so each VM spends less time blocked and more time runnable. Also use a fair-share scheduler (CFS) so a ready VM is dispatched immediately.

Question 6 — Hospital Emergency System Process Scheduling

1. Scheduling issue

Starvation (indefinite blocking) of the lowest-priority process. With strict priority scheduling, as long as P1 (and P2) jobs keep arriving, the CPU is always given to them and P3 never reaches the front of the ready queue.

2. Recommended algorithm

Pre-emptive Priority Scheduling with Aging (implemented as a Multilevel Feedback Queue). Emergency registration remains the highest priority and pre-empts other work; waiting processes slowly gain priority.

3. How starvation is prevented

- **Aging:** raise a waiting process's priority by one level every fixed interval (illustrative: every 30 s of waiting). P3 (priority 3) becomes priority 2 after 30 s and then competes with lab reports; the priority is reset to its base once it runs.
- **Reserved CPU share / time slicing between queues,** e.g. at most 70 % of CPU to the emergency queue, 20 % to lab, 10 % to pharmacy, so every class is guaranteed progress.
- **Cap on boosting:** aged priority can approach but never exceed the emergency level, so P1 is never delayed by aged low-priority jobs.

4. Response time and fairness

- **Response time:** emergency requests still pre-empt immediately, so their response time stays minimal, which is the clinically critical requirement.
- **Fairness and bounded waiting:** every process has a finite upper bound on waiting time; pharmacy billing is completed within a predictable delay rather than being postponed indefinitely.
- **Throughput and satisfaction:** pending bills are cleared, reducing queues at the pharmacy counter without noticeably affecting emergency service.

Question 7 — E-Commerce Warehouse Synchronization

1. Synchronization problem

A **race condition** on shared data: the result depends on the interleaving of unsynchronized read–modify–write operations. Example with stock = 1:

```
T1: read stock (1)
T2: read stock (1)        <- both see the same value
T1: stock = 1 - 1 = 0, confirm order
T2: stock = 1 - 1 = 0, confirm order   <- duplicate confirmation, oversold item
```

One update is lost (stock should be −1 / the second order should have been rejected) and two confirmations are issued for one unit.

2. Why a critical section is required

The code that checks and updates inventory touches shared data and must run **atomically**. A critical section guarantees **mutual exclusion** (only one thread inside at a time), together with progress (a thread outside cannot block others from entering) and bounded waiting (no thread waits forever). Without it, partial updates are visible to other threads and the data become inconsistent.

3. Recommended solution – semaphores

Use a **binary semaphore (mutex)**, initial value 1, per inventory item (per SKU) to allow parallelism across different products:

```
semaphore mutex = 1;          // one per SKU


placeOrder(sku, qty):
   wait(mutex);               // entry section
   if (stock[sku] >= qty) {   // critical section
      stock[sku] -= qty;
      confirm(order);
   } else reject(order);
   signal(mutex);             // exit section
```

4. How data consistency is maintained

- The **check and the decrement are atomic**: the second thread sees the updated stock and is rejected, so no overselling and no duplicate confirmation.
- Only one thread modifies a SKU at a time, so no updates are lost; stock counts always match the number of confirmed orders.
- Always release the semaphore in a `finally` block and keep the critical section short to avoid deadlock and contention; use a unique order ID (idempotency) and database transactions for crash safety.

Question 8 — Video Streaming Service Thread Management

1. Thread-management problem

- **Unbounded thread creation (thread-per-connection):** 60,000 kernel threads, each with its own stack (≈ 1 MB) gives about 60 GB of memory before any data is buffered.
- **Context-switch and scheduler overhead:** the CPU spends its time switching and managing threads instead of encoding/sending video (95 % busy, but little useful work); cache and TLB are thrashed.
- Resulting resource exhaustion causes buffering and possible failures.

2. Suitable multithreading model

Many-to-many (two-level) model with a thread pool. Many user-level connections/tasks are multiplexed onto a small number of kernel threads (about 1–2 per core). Blocking on one connection does not block the others, and the kernel schedules only a limited number of threads.

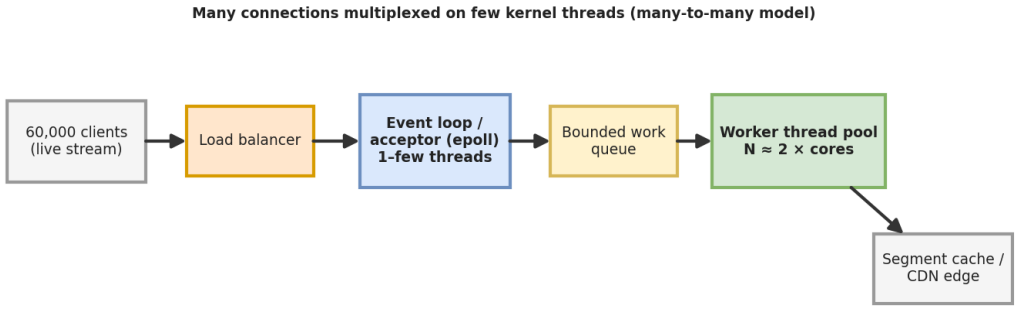


Figure 4 – Proposed architecture (draw.io: q8_threadpool_architecture.drawio)

3. Two optimisations for scalability

- **Bounded thread pool with a work queue:** pool size ≈ cores × (1 + wait time / compute time); excess requests queue (or are shed) instead of creating threads.
- **Non-blocking, event-driven I/O (epoll / io_uring) and caching:** a few event-loop threads watch thousands of sockets; use zero-copy `sendfile`, adaptive-bitrate segments and CDN/edge caches so most clients are served without re-reading or re-encoding.

4. How throughput improves

- Memory drops from ~60 GB of stacks to a few hundred MB, removing swapping.
- Few threads → minimal context switching → the cores spend their time sending data, so more segments per second are delivered and buffering falls.
- Load becomes predictable; the system degrades gracefully under overload via queueing and back-pressure, and scales further by adding servers behind the load balancer.

Question 9 — Smart Traffic Control Deadlock

1. Does a deadlock exist?

Yes. The graph has the cycle T1 → R2 → T2 → R3 → T3 → R4 → T4 → R1 → T1. Each junction is a single-instance resource, so the cycle means all four threads wait forever (gridlock).

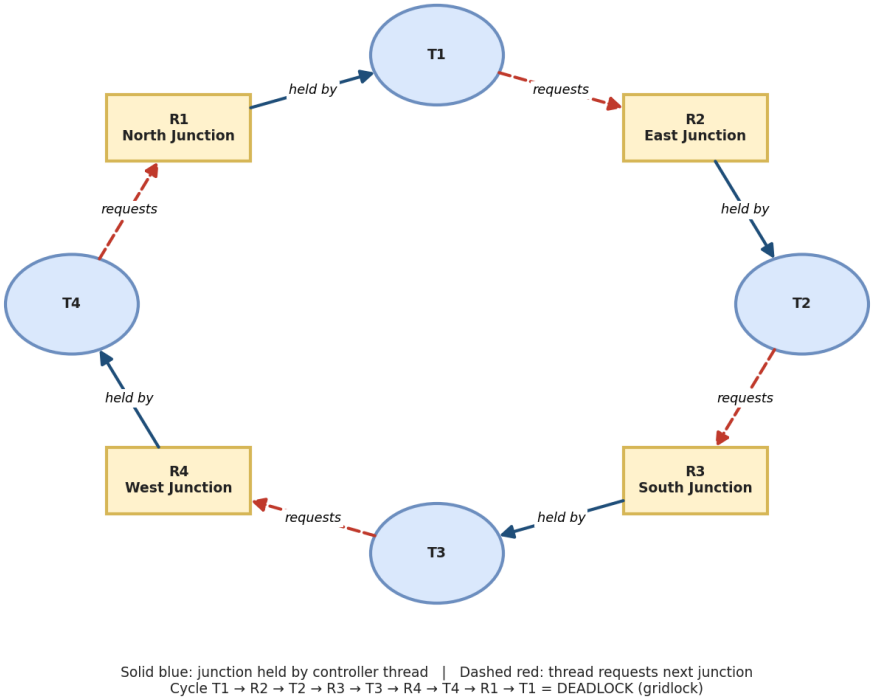


Figure 5 – Resource Allocation Graph (draw.io: q9_rag_traffic.drawio)

2. The four necessary conditions in this scenario

Condition	**In the traffic system**
Mutual exclusion	A junction can be used by only one flow/controller thread at a time.
Hold and wait	Each thread holds one junction (e.g. T1 holds R1) while waiting for the next (R2).
No pre-emption	A junction cannot be forcibly taken from the vehicles/thread that occupy it.
Circular wait	T1 → T2 → T3 → T4 → T1, each waiting for a junction held by the next.

3. Recovery technique

Resource pre-emption with rollback / victim selection. Pick a victim (for example T4, which holds the least-critical flow), force it to release R4 (clear the junction using an all-red phase and reroute/hold its vehicles), and restart it later. T3 then obtains R4 and finishes, releasing R3; T2 proceeds, then T1, and finally T4 retries. Choose the victim by lowest cost (fewest vehicles, lowest priority, least progress) and prevent repeated victimisation by aging.

4. Redesign to prevent recurrence

- **Global resource ordering R1 < R2 < R3 < R4:** every controller must request junctions in increasing order, so T4 must request R1 before R4. A circular wait is impossible.
- **All-or-nothing (atomic) reservation:** a central coordinator grants a controller all junctions of a route at once or none, removing hold-and-wait.
- Add timeouts on waits, a monitor for wait-for cycles, and coordinated ‘green-wave’ signal plans to keep junction queues from spilling back.

Question 10 — University Examination Server Resource Allocation

1. Applying the Banker's Algorithm

Each student session is a 'customer' and CPU slots, memory blocks and DB connections are the resource types. When a session starts it declares its **maximum** requirement. For every later request the server **simulates** the allocation and grants it only if the resulting state is **safe**; otherwise the session waits. Resources are never allocated into a state from which all sessions cannot finish, so deadlock is avoided, while everything safe is still allocated (high utilization, unlike prevention that restricts requests up-front).

2. Information required before granting a request

- **Available[m]:** free units of each resource type.
- **Max[n][m]:** maximum demand declared by each session.
- **Allocation[n][m]:** units currently held by each session.
- **Need[n][m] = Max − Allocation.**
- **Request[i][m]:** the new request. Checks: Request ≤ Need (not above its declared maximum) and Request ≤ Available (else wait).

3. Determining a safe state

Tentatively set Available −= Request, Allocation += Request, Need −= Request, then run the **safety algorithm**: (1) Work = Available, Finish[i] = false. (2) Find a session i with Finish[i] = false and Need[i] ≤ Work. (3) Work += Allocation[i], Finish[i] = true; repeat. (4) If all sessions can finish the state is **safe** (a safe sequence exists) and the request is granted; otherwise it is rolled back and the session waits.

Example (5 sessions, 3 resource types, Available = (3, 3, 2)):

Session	Allocation	Max	Need	Work after finishing
S0	0 1 0	7 5 3	7 4 3	7 5 5
S1	2 0 0	3 2 2	1 2 2	5 3 2
S2	3 0 2	9 0 2	6 0 0	10 5 7
S3	2 1 1	2 2 2	0 1 1	7 4 3
S4	0 0 2	4 3 3	4 3 1	7 4 5

Safe sequence found: ⟨S1, S3, S4, S0, S2⟩, so the state is safe.

4. Limitation in large-scale cloud systems

Scalability and the need for advance knowledge. The algorithm requires every process to declare its maximum demand beforehand, and a fixed, known set of processes/resources; in the cloud demands are dynamic, workloads are elastic and resources are distributed across many nodes. The safety check costs $O(m \cdot n^2)$ for each request, which is too slow and needs a global view of the state with thousands of concurrent sessions, making Banker's algorithm impractical there (and it is conservative, so declared maxima that are over-estimated leave resources idle).

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand